1v1 CODING DUEL

AGILE SCRUM · SIX-WEEK DELIVERY PLAN

A practical sprint roadmap for a three-person development team

Project: Real-time JavaScript coding competition for two players

Team: 3 members **Duration:** 6 weeks **Method:** Scrum

Purpose of this document

This plan converts the supplied README and project plan into three two-week Scrum sprints. Every sprint has one shared goal and clearly assigned contributions for all three team members. The scope remains focused on the project's stated MVP: room creation and joining, a synchronized coding duel, JavaScript test execution, hints, results, and online deployment.

Planning assumption: The project plan specifies six weeks. This document therefore uses three two-week sprints. The README describes a more complete implementation; features are scheduled only where they fit the six-week MVP, and optional upgrades remain outside the committed scope.

1. Project Overview and Scrum Setup

Product vision

Build a simple, reliable web application where two players join the same room using a six-character code, receive the same JavaScript challenge, race against a server-controlled timer, run code against test cases, and see a clear winner or draw.

MVP scope taken from the project files

• Create a room with a unique six-character code; join with a nickname and code, without user accounts.

• Show the same coding problem to both players and provide a browser-based JavaScript editor.

• Run tests in a Web Worker with a roughly two-second execution timeout and display passed/failed results.

• Keep the match timer and winner decision authoritative on the server; show hints at fixed time points.

• Handle winner, draw, invalid/full rooms, empty nicknames, and mid-game disconnections.

• Deploy to a host that supports WebSockets, such as Render or Railway, and validate with two separate clients.

• Provide at least six usable problems by completion, with a stretch goal of 8–10 if time permits.

Technology baseline

Area	Planned technology / responsibility
Frontend	React + Vite + Tailwind CSS; screens for home, lobby, duel, and game over.
Editor	CodeMirror 6 or Monaco Editor; choose one early and avoid switching mid-project.
Backend and real-time	Node.js, Express, Socket.IO; rooms, timer, hints, authoritative results.
Code runner	Browser Web Worker with timeout and visible test feedback.
Problem data	problems.json with title, difficulty, time limit, description, examples, hints, and tests.
Tests and deployment	Automated integration tests; Render or Railway with WebSocket support.

Scrum roles for this small team

Role	Who / how it works
Product Owner (PO)	One team member acts as PO for backlog ordering and acceptance. Rotate the meeting facilitator if helpful; the PO remains responsible for clarifying what matters most.
Scrum Master / facilitator	A team member facilitates planning, daily Scrum, review, and retrospective, and helps remove blockers. This can rotate by sprint.
Developers	All three members build, test, review, and integrate the product. The role labels below indicate primary ownership, not isolated silos.

Recommended working agreements

• Work in small Git branches; open pull requests and ask another member to review before merging.

• Integrate at least daily. Agree on Socket.IO event names and payload shapes before parallel implementation.

• Keep a visible task board: Product Backlog → Sprint Backlog / To Do → In Progress → Review → Done.

• The Daily Scrum is a 15-minute coordination meeting, not a status report to a manager.

2. Sprint 1 — Foundation and Multiplayer Rooms

Duration: Weeks 1–2 **Sprint goal:** Two players can create and join the same room, see lobby status, and enter a stable shared match flow.

Sprint backlog and ownership

Team member	Primary tasks	Deliverables / acceptance checks
Member 1 — Backend & real-time	Initialize Node.js/Express/Socket.IO server; define room data model; generate unique six-character codes; implement create_room and join_room; validate nickname, room existence, capacity, and match state.	Server starts locally; create_room returns a unique code; valid second player joins; invalid/full room requests return clear errors; room state tracks two players.
Member 2 — Frontend & UX	Set up React + Vite + Tailwind; create the home screen for nickname/create/join; create lobby screen with room code and player slots; wire socket client; show loading and error states.	A player can create a room from the UI; another can join using code; lobby displays both nicknames / connection status; layout works at desktop and mobile widths.
Member 3 — Problem data, integration & QA	Agree on the problem JSON schema; prepare the first 3 sample JavaScript problems; write a manual test checklist; test room creation/joining with two browser tabs; document event payloads and local setup.	problems.json follows the agreed schema; first 3 problems reviewed; two-client room flow tested; setup instructions and event contract committed to the repository.

Sprint 1 stories

• **US-01:** As a player, I can create a room and receive a shareable six-character code.

• **US-02:** As a second player, I can join an existing room using its code and a nickname.

• **US-03:** As a player, I can see whether the room is waiting for an opponent or has two players.

• **US-04:** As a player, I receive a clear message if the nickname or room code is invalid, or the room is full.

Sprint 1 review demo

Run the application locally in two browser tabs or on two devices. Create a room in one, join from the other, confirm both clients see the same room and player status, then demonstrate the error states.

Sprint 1 exit criteria

• The app starts from documented setup steps on another team member’s machine.

• Room creation/joining works with two separate socket clients and no duplicate room-code collision in normal testing.

• The frontend and backend use one agreed event/payload contract.

• The Product Backlog is refined for Sprint 2, with editor and runner choices finalized.

3. Sprint 2 — Coding Challenge and Match Logic

Duration: Weeks 3–4 **Sprint goal:** Both players receive the same challenge, run JavaScript tests safely in the browser, and play a timed match with hints and a correct result.

Sprint backlog and ownership

Team member	Primary tasks	Deliverables / acceptance checks
Member 1 — Backend & game rules	Implement game_start and synchronized countdown; server-authoritative timer and timer_tick; difficulty-based limits (Easy 5 min, Medium 10 min, Hard 15 min); hint dispatch at 1/3 and 2/3; first-solved winner decision; draw and disconnect-forfeit logic; game_over event.	Both clients receive the same problem and timer state; timer is controlled by server; hints appear at agreed points; only first accepted solved event wins; timeout yields draw; disconnect rule is handled.
Member 2 — Frontend & editor	Build countdown overlay and duel screen; integrate CodeMirror or Monaco; display problem statement/examples, timer, hints, opponent progress, Run & Test controls, test feedback, and game-over result; wire socket events.	Both clients see the same problem; editor accepts JavaScript; timer and hints update in UI; test results are readable; win/draw/defeat states render correctly.
Member 3 — Runner, problems & tests	Build Web Worker runner with a 2-second timeout; execute solve function against test cases; compare actual vs expected values; show passed/failed cases and console output where feasible; expand problem bank to at least 6; write integration tests for timer, solved, draw, and disconnect.	Infinite-loop sample is terminated without freezing the page; correct/incorrect solutions show expected results; minimum 6 problems have valid test data; integration tests cover core match paths.

Sprint 2 stories

- **US-05:** When the second player joins, both players receive a synchronized countdown and the same challenge.
- **US-06:** As a player, I can write a JavaScript solve function and run it against the challenge's tests.
- **US-07:** As a player, I see passed/failed test cases and useful expected-versus-actual feedback.
- **US-08:** The match ends with a winner when all tests pass first, or a draw when the timer expires.
- **US-09:** Hints unlock at fixed points in the match; a disconnected opponent triggers the agreed forfeit outcome.

Important integration contract

Agree on event names and payloads before implementing in parallel. The supplied plan names client-to-server events create_room, join_room, and solved; server-to-client events include room_created, player_joined, game_start, timer_tick, hint, and game_over. Add events only when needed and document them in one shared file. The server, not the browser, must make the final winner/timer decision.

Sprint 2 review demo

Play a complete duel with two clients: countdown → shared problem → run correct and incorrect code → show progress and hints → finish with a win or draw. Demonstrate that an infinite loop times out and the page remains responsive.

4. Sprint 3 — Hardening, Deployment and Final Demo

Duration: Weeks 5–6 **Sprint goal:** Deliver a stable, hosted MVP that works for two players on separate devices and is ready for demonstration.

The source project plan places hosting and polish in Week 4. In this six-week Scrum plan, deployment begins as soon as the core game is stable, then Sprint 3 focuses on verification, fixes, and release readiness.

Sprint backlog and ownership

Team member	Primary tasks	Deliverables / acceptance checks
Member 1 — Backend reliability & deployment	Prepare production server configuration; verify PORT handling and WebSocket support; harden room lifecycle and duplicate/simultaneous solved handling; review disconnection, timeout, invalid room, and rematch behavior if rematch is in MVP; assist production deployment.	Hosted server starts reliably; room state transitions are consistent; simultaneous submissions produce one winner; logs help diagnose failures; no secrets or machine-specific paths are committed.
Member 2 — UI polish & responsive behavior	Fix UI issues from testing; improve empty/loading/error states, timer visibility, result screen, sound toggle if included, and responsive layout; verify direct share/join link behavior if implemented.	Core screens are usable on desktop and mobile; errors are understandable; the full match flow can be completed without developer tools; UI feedback matches server state.
Member 3 — QA, regression & release material	Run automated test suite and two-device exploratory tests; verify all problems and edge cases; prepare README/demo script, known-limitations note, screenshots, and presentation; retest fixes before release.	Regression checklist passes; at least 6 problems are playable; deployment steps and live URL are documented; demo is rehearsed; known client-side test-case/cheating limitation is disclosed.

Sprint 3 stories

• **US-10:** As a player, I can play with a friend from separate devices through the hosted application.

• **US-11:** As a player, I receive stable behavior and clear feedback when a room is invalid, full, or a connection drops.

• **US-12:** As a project evaluator, I can follow setup instructions, see the main features demonstrated, and understand known limitations.

Sprint 3 review demo

Use two separate devices or networks to create and join a room on the deployed URL. Complete a coding challenge, demonstrate a failed attempt followed by a successful solution, then show draw/disconnection behavior where feasible. Finish by presenting the architecture, tests, and known limitations.

Release checklist

• Two players can create/join a room on different devices.

• Both players receive the same challenge and synchronized server-owned timer.

• Run & Test produces correct feedback; infinite loops are stopped after the timeout.

• Hints, winner, draw, and disconnection outcomes work as specified.

• At least six problems are available and tested.

• Hosted application works from outside the developers' local network.

• README includes setup, scripts, deployment, demo steps, and the known limitation that client-side tests can be inspected or manipulated.

5. How the Team Uses Scrum

Sprint cadence

Ceremony	When	Purpose / output
Sprint Planning	First day of each 2-week sprint; 60–90 min	Agree on a realistic Sprint Goal, select backlog items, split each item into tasks, assign a primary owner and reviewer.
Daily Scrum	Every working day; 15 min	Each member shares progress toward the Sprint Goal, next step, and blocker. Update the task board immediately after.
Backlog Refinement	Once mid-sprint; 30–45 min	Clarify upcoming stories, acceptance criteria, dependencies, and estimates. Do not over-plan low-priority extras.
Sprint Review	Last day; 30–45 min	Demo working software to teammates/stakeholders, compare it with acceptance criteria, collect feedback, and update the Product Backlog.
Sprint Retrospective	After review; 30 min	Discuss what worked, what slowed the team down, and one or two concrete improvements for the next sprint.

Artifacts

• **Product Backlog:** ordered list of user stories, defects, and technical tasks; the Product Owner reorders it as new information appears.

• **Sprint Backlog:** selected stories plus the plan for delivering them in the current sprint; the team updates it daily.

• **Increment:** integrated, tested, usable software at the end of each sprint—not merely code that works on one person's machine.

• **Definition of Done:** shared quality bar used to decide whether work is truly complete.

Suggested Definition of Done

• Acceptance criteria are met and the feature works in the integrated application.

• Code is committed to Git, reviewed by another team member, and merged without breaking the main branch.

• Relevant manual or automated tests pass; edge cases are checked where applicable.

• UI states and error messages are understandable; no known blocker remains for the feature.

• README or technical notes are updated when setup, events, or behavior changes.

Estimating and task sizing

Use story points or simple T-shirt sizes if the team is new to estimation. Keep tasks small enough to finish in roughly half a day to two days. Estimate collaboratively; avoid assigning a large story to one person without splitting out integration and testing work.

6. Product Backlog, Dependencies and Risks

Prioritized backlog

Priority	Backlog item	Sprint target
P0 — Must	Room creation, unique code, join by code/nickname, room validation	Sprint 1
P0 — Must	Problem schema and shared problem delivery to both players	Sprint 1–2
P0 — Must	Editor + Web Worker runner + test feedback + timeout	Sprint 2
P0 — Must	Server-authoritative timer, hints, winner/draw and disconnection handling	Sprint 2
P0 — Must	At least 6 problems, integrated two-client testing, online deployment	Sprint 2–3
P1 — Should	Progress indicator, improved sound and visual polish, shareable join URL	Sprint 2–3 if core is stable
P2 — Could / stretch	Rematch flow, penalty seconds, spectator mode, accounts/history, more languages, server-side rechecking	Only after MVP is done

Key dependencies

• Frontend and backend agree on socket event names, payload fields, and error shapes before integration.

• Problem JSON schema must be agreed before the runner and problem bank are developed independently.

• The winner decision and timer are server-authoritative; browser-side test execution is not a secure anti-cheat system.

• Deployment requires a Node service that supports WebSockets; verify hosting behavior early rather than at the final hour.

Risk register and response

Risk	Impact	Mitigation / owner
Socket event mismatch	Integration delays and broken match flow	Write a shared event contract in Sprint 1; Member 1 owns backend definitions; all members review changes.
Editor/runner takes longer than expected	Core duel cannot be demonstrated	Choose CodeMirror or Monaco in Sprint 1; Member 3 prototypes the worker early; keep JavaScript-only scope.
Timer drift or duplicate winner	Unfair or inconsistent outcomes	Keep timer/winner state on server; test near-simultaneous solved events and timeout boundary.
Infinite loop freezes UI	Browser becomes unresponsive	Run submitted code in a Web Worker and terminate it after the defined timeout.
Hosting/WebSocket issue	Remote play fails	Deploy a minimal socket connection early; confirm platform WebSocket support and PORT configuration.
Scope creep	MVP unfinished by week six	Keep accounts, spectator mode, more languages, and server-side judging as stretch goals only.

7. Six-Week Calendar at a Glance

Week	Focus	Member 1	Member 2	Member 3
1	Sprint 1 — setup and room protocol	Server setup, room model, code generation	React/Vite setup, home screen	Schema draft, initial problems, test checklist
2	Sprint 1 — complete room flow	Validation and socket room events	Lobby and socket integration	Two-client tests, event contract, review support
3	Sprint 2 — challenge and runner	Countdown, timer foundation, game events	Duel screen and editor integration	Web Worker runner, test feedback, expand problems
4	Sprint 2 — finish match logic	Hints, winner/draw/disconnect logic	Timer/hint/result UI, opponent status	Integration tests, six-problem minimum, edge cases
5	Sprint 3 — deploy and harden	Production config, room-state hardening	Responsive UI and usability fixes	Regression tests, deploy verification, docs draft
6	Sprint 3 — release and demo	Fix backend defects, support final release	Final UI fixes and demo flow	Final QA, README, demo/presentation, known limitations

Team-wide responsibilities every sprint

• All members pull the latest main branch, run the app, and review integration at least once per working day.

• The primary owner is accountable for moving a task forward; the whole team remains responsible for quality and delivery.

• A task is not Done until it is integrated and verified, not just implemented locally.

• At the end of each sprint, show a working increment and use the retrospective to adjust the process.

Final recommendation

Protect the MVP and integrate continuously. The highest-risk parts are real-time synchronization, safe code execution, and deployment with WebSockets. Validate those paths early, then spend the final sprint fixing defects and proving that two people can play from separate devices.

Source basis: the uploaded README.md and plan(2).md. The sprint structure, team ownership, ceremonies, and planning recommendations are the proposed Scrum organization of the features and six-week duration described in those files.